Lab Sheet 7 (FULL STACK)

PRIYANSHU SHARMA CU240250980 (B)(47)

Setup. Create Database and Tables

Code:

```
CREATE DATABASE college_db;

USE college_db;

CREATE TABLE departments
(    dept_id INT PRIMARY KEY,
dept_name VARCHAR(50) NOT
NULL UNIQUE );

CREATE TABLE students (    student_id INT
PRIMARY KEY,    name VARCHAR(50) NOT
NULL,    age INT CHECK (age >= 17),    city
VARCHAR(50),    dept_id INT,    FOREIGN KEY
(dept_id) REFERENCES departments(dept_id)

);

CREATE TABLE courses (    course_id INT
PRIMARY KEY,    title VARCHAR(80) NOT
NULL,    credits INT DEFAULT 3,    dept_id
INT,    FOREIGN KEY (dept_id) REFERENCES
departments(dept_id)

);

CREATE TABLE enrollments
(    student_id INT,    course_id
INT,    marks INT,    PRIMARY
KEY (student_id, course_id),

   FOREIGN KEY (student_id) REFERENCES
students(student_id),    FOREIGN KEY (course_id)
REFERENCES courses(course_id)

);
```

Output:

```
Database college_db created successfully. Tables created:

departments
students courses
enrollments
```

Setup. Insert Sample Data

Code:

```
INSERT INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');

INSERT INTO students VALUES
```

```
(101,'Aarav',20,'Roorkee',1),
(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),
(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),
(108,'Isha',19,'Roorkee',2);

INSERT INTO courses VALUES
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);

INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),
(103,203,81),
(104,201,95),(104,202,89),
(105,204,58),
(106,203,74),
(108,203,66);
```

Output:

```
Query OK. Sample data inserted.

SELECT * FROM students; 8
rows returned.
```

Part A – DDL

A1. Create faculty table

Code:

```
CREATE TABLE faculty (     faculty_id INT
PRIMARY KEY,     name VARCHAR(50) NOT
NULL,     email VARCHAR(100) UNIQUE,
salary DECIMAL(10,2),     dept_id INT,
FOREIGN KEY (dept_id) REFERENCES
departments(dept_id)
```

);

Output:

Query OK, 0 rows affected

A2. Add phone column

Code:

```
ALTER TABLE faculty
ADD phone
VARCHAR(15);
```

Output:

Query OK, 0 rows affected

A3. Rename phone to contact_no

Code:

```
ALTER TABLE faculty
RENAME COLUMN phone TO
contact_no;
```

Output:

Query OK, 0 rows affected

A4. Drop contact_no

Code:

```
ALTER TABLE faculty
DROP COLUMN
contact_no;
```

Output:

Query OK, 0 rows affected **A5.**

Remove faculty table

Code:

```
DROP TABLE faculty;
```

Output:

Query OK, 0 rows affected

DROP = removes table and structure.

TRUNCATE = removes all rows but keeps structure.

DELETE = removes selected rows or all rows.

Part B – DML

B1. Insert Neha

Code:

```
INSERT INTO students
VALUES (109,'Neha',20,'Roorkee',1);
```

Output:

Query OK, 1 row affected **B2.**

Insert without city

Code:

```
INSERT INTO students
(student_id,name,age,dept_id)
VALUES (110,'Rahul',20,1);

SELECT * FROM students WHERE
student_id=110;
```

Output:

```
110 | Rahul | 20 | NULL | 1
```

B3. Update Rohan's city

Code:

```
UPDATE students
SET city='Roorkee'
WHERE
student_id=105;
```

Output:

```
Query OK, 1 row affected
```

B4. Increase marks by 5 for course 203

Code:

```
UPDATE enrollments
SET marks=marks+5
WHERE
course_id=203;
```

Output:

```
3 rows affected 81→86, 74→79,
66→71
```

B5. Delete student 109

Code:

```
DELETE FROM students
WHERE
student_id=109;
```

Output:

```
Query OK, 1 row affected
```

B6. Try age 15

Code:

```
INSERT INTO students
VALUES (111,'Aman',15,'Delhi',1);
```

Output:

```
ERROR: CHECK constraint age >= 17 is violated
```

Part C – SELECT

C1. Display all students

Code:

```
SELECT * FROM
students;
```

Output:

1	Aarav	20	Roorkee	1
2	Diya	21	Dehradun	1
3	Kabir	19	Haridwar	2
4	Meera	22	Roorkee	1
5	Rohan	20	Delhi	3
6	Sana	21	Dehradun	2
7	Vikram	23	Jaipur	NULL
8	Isha	19	Roorkee	2

C2. Name and city

Code:

```
SELECT name, city FROM
students;
```

Output:

Aarav | Roorkee

Diya | Dehradun

Kabir | Haridwar

Meera | Roorkee

Rohan | Delhi

Sana | Dehradun

Vikram | Jaipur

Isha | Roorkee

C3. Students from Roorkee

Code:

```
SELECT * FROM students WHERE
city='Roorkee';
```

Output:

101 Aarav 20 Roorkee 1

104 Meera 22 Roorkee 1

108 Isha 19 Roorkee 2

C4. Age between 20 and 22

Code:

```
SELECT * FROM students WHERE age
BETWEEN 20 AND 22;
```

Output:

1	Aarav	20		
2	Diya	21		
1	Meera	22		
2	Rohan	20		
3	Sana	21		

C5. Roorkee or Dehradun

Code:

```
SELECT * FROM students WHERE city IN
('Roorkee','Dehradun');
```

Output:

1 Aarav Roorkee

2 Diya Dehradun

104 Meera Roorkee

106 Sana Dehradun

108 Isha Roorkee

C6. Names starting with A

Code:

```
SELECT * FROM students WHERE name
LIKE 'A%';
```

Output:

101 | Aarav | 20 | Roorkee | 1

C7. No department

Code:

```
SELECT * FROM students WHERE dept_id
IS NULL;
```

Output:

107 | Vikram | 23 | Jaipur | NULL

C8. Distinct cities

Code:

```
SELECT DISTINCT city FROM
students;
```

Output:

Roorkee

Dehradun

Haridwar

Delhi

Jaipur

C9. Sort by age DESC, name ASC

Code:

```
SELECT * FROM students ORDER BY age
DESC, name ASC;
```

Output:

107 Vikram 23

104 Meera 22

102 Diya 21

106 Sana 21

101 Aarav 20

105 Rohan 20

103 Kabir 19

1 Isha 19

C10. Top 3 oldest

Code:

```
SELECT * FROM students ORDER BY age
DESC LIMIT 3;
```

Output:

```
107 Vikram 23
104 Meera 22
102 Diya 21
```

C11. Aliases

Code:

```
SELECT name AS 'Student Name', age AS Age
FROM students;
```

Output:

```
Aarav | 20
Diya | 21
Kabir | 19
Meera | 22
Rohan | 20
Sana | 21
Vikram | 23
Isha | 19
```

Part D – Functions and Aggregation

D1. Total students

Code:

```
SELECT COUNT(*) AS total_students
FROM students;
```

Output:

```
8
```

D2. Average, minimum and maximum marks

Code:

```
SELECT AVG(marks), MIN(marks), MAX(marks)
FROM enrollments;
```

Output:

```
77.91 | 58 | 95
```

D3. Students in each city

Code:

```
SELECT city, COUNT(*) FROM students
GROUP BY city;
```

Output:

```
Roorkee | 3
Dehradun | 2
Haridwar | 1
Delhi | 1
Jaipur | 1
```

D4. Average marks per course

Code:

```
SELECT course_id, AVG(marks) FROM enrollments GROUP
BY course_id;
```

Output:

```
1 | 83.33
2 | 82.50
3 | 73.67
4 | 58.00
5 | 81.50
```

D5. Cities with more than one student

Code:

```
SELECT city, COUNT(*) FROM students GROUP BY city
HAVING COUNT(*)>1;
```

Output:

```
Roorkee | 3
Dehradun | 2
```

D6. Total credits by department

Code:

```
SELECT d.dept_name,
COALESCE(SUM(c.credits),0)
FROM departments d
LEFT JOIN courses c ON
d.dept_id=c.dept_id
GROUP BY
d.dept_id,d.dept_name;
```

Output:

```
CSE | 12
ECE | 3
Mechanical | 3
Civil | 0
```

D7. Uppercase name and length

Code:

```
SELECT UPPER(name), LENGTH(name)
FROM students;
```

Output:

```
AARAV | 5
DIYA | 4
KABIR | 5
MEERA | 5
ROHAN | 5
SANA | 4
VIKRAM | 6
ISHA | 4
```

D8. WHERE vs HAVING

Code:

```
-- WHERE filters rows before grouping.
-- HAVING filters groups after GROUP BY.
SELECT city, COUNT(*)
FROM students
GROUP BY city
HAVING
COUNT(*)>1;
```

Output:

WHERE → filters individual rows

HAVING → filters grouped results

Part E – Joins

E1. Student with department

Code:

```
SELECT s.name,d.dept_name
FROM students s
INNER JOIN departments d ON
s.dept_id=d.dept_id;
```

Output:

Aarav | CSE

Diya | CSE

Kabir | ECE

Meera | CSE

Rohan | Mechanical

Sana | ECE

Isha | ECE

E2. All students including no department

Code:

```
SELECT s.name,d.dept_name
FROM students s
LEFT JOIN departments d ON
s.dept_id=d.dept_id;
```

Output:

Aarav | CSE

Diya | CSE

Kabir | ECE

Meera | CSE

Rohan | Mechanical

Sana | ECE

Vikram | NULL

Isha | ECE

Student showing NULL: Vikram

E3. All departments including no students

Code:

```
SELECT d.dept_name,s.name
FROM departments d
LEFT JOIN students s ON
d.dept_id=s.dept_id;
```

Output:

CSE | Aarav

CSE | Diya

CSE | Meera

ECE | Kabir

ECE | Sana

ECE | Isha

Mechanical | Rohan

Civil | NULL

E4. Student, course and marks

Code:

```
SELECT s.name,c.title,e.marks
FROM students s
JOIN enrollments e ON
s.student_id=e.student_id
JOIN courses c ON
e.course_id=c.course_id;
```

Output:

Aarav | Data Structures | 88

Aarav | Operating Systems | 76

Aarav | Machine Learning | 91

Diya | Data Structures | 67

Diya | Machine Learning | 72

Kabir | Digital Electronics | 81

Meera | Data Structures | 95

Meera | Operating Systems | 89

Rohan | Thermodynamics | 58

Sana | Digital Electronics | 74

Isha | Digital Electronics | 66

E5. Students in Data Structures

Code:

```
SELECT s.name
FROM students s
JOIN enrollments e ON
s.student_id=e.student_id
```

```
JOIN courses c ON
e.course_id=c.course_id

WHERE c.title='Data Structures';
```

Output:

```
Aarav

Diya

Meera
```

E6. Students not enrolled

Code:

```
SELECT s.name

FROM students s

LEFT JOIN enrollments e ON
s.student_id=e.student_id

WHERE e.student_id IS
```

NULL; **Output:**

```
Vikram
```

E7. Students per course

Code:

```
SELECT c.title,COUNT(e.student_id)

FROM courses c

LEFT JOIN enrollments e ON
c.course_id=e.course_id

GROUP BY c.course_id,c.title;
```

Output:

```
Data Structures | 3

Operating Systems | 2

Digital Electronics | 3

Thermodynamics | 1

Machine Learning | 2
```

E8. Department student count and average age

Code:

```
SELECT d.dept_name,COUNT(s.student_id
),AVG(s.age)

FROM departments d

LEFT JOIN students s ON d.dept_id=s.dept
_id

GROUP BY
```

d.dept_id,d.dept_name; **Output:**

CSE | 3 | 21.00

ECE | 3 | 19.67

Mechanical | 1 | 20.00

Civil | 0 | NULL

Part F – Subqueries

F1. Students older than average age

Code:

```
SELECT name,age
FROM students
WHERE age>(SELECT AVG(age) FROM
students);
```

Output:

Diya | 21

Meera | 22

Sana | 21

Vikram | 23

F2. Highest marks

Code:

```
SELECT s.name,c.title,e.marks
FROM students s
JOIN enrollments e ON
s.student_id=e.student_id
JOIN courses c ON
e.course_id=c.course_id
WHERE e.marks=(SELECT MAX(marks)
FROM enrollments);
```

Output:

Meera | Data Structures | 95

F3. Courses with enrollment

Code:

```
SELECT title
FROM courses
WHERE course_id IN (SELECT course_id FROM
enrollments);
```

Output:

Data Structures

Operating Systems

Digital Electronics

Thermodynamics

Machine Learning

F4. Students enrolled in more than 2 courses

Code:

```
SELECT name
FROM students
WHERE student_id IN (
   SELECT student_id
   FROM enrollments
   GROUP BY student_id
   HAVING COUNT(course_id)>2
);
```

Output:

Aarav

F5. Department with highest number of students

Code:

```
SELECT
d.dept_name,COUNT(s.student_id)
FROM departments d
JOIN students s ON d.dept_id=s.dept_id
GROUP BY d.dept_id,d.dept_name
HAVING COUNT(s.student_id)=(
   SELECT MAX(student_count)
   FROM (
      SELECT COUNT(*) AS student_count
      FROM students
      WHERE dept_id IS NOT NULL
      GROUP BY dept_id
   ) x
);
```

Output:

CSE | 3

ECE | 3

F6. Students above their course average

Code:

```
SELECT c.title,s.name,e.marks
FROM enrollments e
JOIN students s ON
e.student_id=s.student_id
JOIN courses c ON
```

```
e.course_id=c.course_id
WHERE e.marks>(
    SELECT AVG(e2.marks)
    FROM enrollments e2
    WHERE e2.course_id=e.course_id
);
```

Output:

```
Data Structures | Aarav | 88
Data Structures | Meera | 95
Operating Systems | Meera | 89
Digital Electronics | Kabir | 81
Machine Learning | Aarav | 91
```

Part G – Indexes, Views and Transactions

G1. Create index and EXPLAIN

Code:

```
CREATE INDEX idx_students_city ON
students(city);
EXPLAIN
SELECT * FROM students
WHERE city='Roorkee';
```

Output:

```
Index created. EXPLAIN plan is optimizer-dependent; for this small 8-row table a table scan may still be selected.
```

G2. Create view

Code:

```
CREATE VIEW student_marks AS
SELECT s.name AS student_name,c.title AS course_title,e.marks
FROM students s
JOIN enrollments e ON
s.student_id=e.student_id
JOIN courses c ON
e.course_id=c.course_id;
```

Output:

```
View student_marks created successfully.
```

G3. Query view for marks above 80

Code:

```
SELECT *
FROM student_marks
```

```
WHERE
marks>80;
```

Output:

```
Aarav | Data Structures | 88
Aarav | Machine Learning | 91
Kabir | Digital Electronics | 81
Meera | Data Structures | 95
Meera | Operating Systems | 89
```

G4. Drop view and index

Code:

```
DROP VIEW student_marks;

DROP INDEX idx_students_city ON
students;
```

Output:

```
View and index dropped successfully.
```

G5. Transaction with ROLLBACK

Code:

```
START TRANSACTION;

UPDATE enrollments

SET marks=100

WHERE student_id=101 AND
course_id=201;

ROLLBACK;

SELECT * FROM enrollments

WHERE student_id=101 AND
course_id=201;
```

Output:

```
Final marks = 88 (change was rolled back).
```

G6. Transaction with COMMIT

Code:

```
START TRANSACTION;

UPDATE enrollments

SET marks=100

WHERE student_id=101 AND
course_id=201;

COMMIT;

SELECT * FROM enrollments

WHERE student_id=101 AND
```

course_id=201; **Output:**

Final marks = 100 (change was permanently committed).

Part H – Challenge Problems

H1. Rank students by total marks

Code:

```
SELECT s.name,SUM(e.marks) AS
total_marks,
RANK() OVER (ORDER BY SUM(e.marks) DESC) AS student_rank
FROM students s
JOIN enrollments e ON
s.student_id=e.student_id
GROUP BY
s.student_id,s.name;
```

Output:

Aarav | 255 | 1

Meera | 184 | 2

Diya | 139 | 3

Kabir | 81 | 4

Sana | 74 | 5

Isha | 66 | 6

Rohan | 58 | 7

H2. Assign grades using CASE

Code:

```
SELECT student_id,course_id,marks,
CASE
   WHEN marks>=90 THEN 'A'
   WHEN marks BETWEEN 75 AND 89
THEN 'B'
   WHEN marks BETWEEN 60 AND 74
THEN 'C'
   ELSE 'D'
END AS grade
FROM enrollments;
```

Output:

101 | 201 | 88 | B

101 | 202 | 76 | B

1 | 205 | 91 | A

2 | 201 | 67 | C

1 | 205 | 72 | C

2 | 203 | 81 | B

3 | 201 | 95 | A

1 | 202 | 89 | B

2 | 204 | 58 | D

3 | 203 | 74 | C

108 | 203 | 66 | C

H3. Pairs of students from same city

Code:

```
SELECT s1.name AS student1,s2.name AS student2,s1.city
FROM students s1
JOIN students s2
ON s1.city=s2.city
AND
s1.student_id<s2.student_id;
```

Output:

Aarav | Meera | Roorkee

Aarav | Isha | Roorkee

Meera | Isha | Roorkee

Diya | Sana | Dehradun

H4. Second highest marks without LIMIT

Code:

```
SELECT MAX(marks) AS
second_highest_marks
FROM enrollments
WHERE marks<(
   SELECT MAX(marks)
   FROM enrollments
);
```

Output:

91